

Romi Lesson 18 Update to RomiBot2026

At this point, you probably have about 90% of what you need to know to program the competition robot. The lessons up to this point have used the RomiReference code as a starting point. This code was written originally several years ago and as you have seen, it includes many cases of deprecated functions and other issues. From this point forward we will start our lessons with a more up to date program. This code is much closer to the competition configuration than we have been using.

You need to clone RomiBot2026 from GitHub onto your computer.

(<https://github.com/team294/RomiBot2026>) Once you do that, open a new branch with your name on it which will be your workspace.

There are several new concepts in this code that you need to get familiar with. This code is compatible with the 2026 FRC competition code. (There will be some changes for the 2027 season since the RoboRio is being replaced, but these are not available as of this writing.)

A subsystem LED.java was added to control Leds separate from OnBoardIO. This enables subsystem methods like LED.get() etc. which were not available in the RomiRreference code OnBoardIO which we had been using.

Data Logging is an important function for improving and troubleshooting the robot. By reviewing the robot log after a match or practice run, we gain insight into how the systems performed and how they can be improved.

Several other utilities are added to add some special math functions and data logging functions that are not included in wpilib.

The code is more robust and readable with better adherence to putting all fixed constants in the constants class.

Read through this code to be familiar with all the classes and structure.

SmartDashboard - Elastic

So far we have been using the joysticks and joystick buttons to do all of our controlling of the robot. In actual practice, the robot is controlled from a driver station which is a combination of joysticks, switches, and a SmartDashboard which is a virtual control panel displayed on the driver station laptop. The SmartDashboard is very powerful because it can have many tabs and many functions

and only has to display the functions that are useful for the particular state of the system. There may be a tab for testing and another for calibration and another for trouble shooting. Usually the tab used during a competition would include among other functions the menu to select which Autonomous routine is being used as well as other unique functions. In this lesson we will learn how to set up this dashboard.

Look at RobotContainer.java and find the 'configureDashboard' method. This is where the Autonomous routine to be used is selected. You have already used similar code to select which autonomous methods are used, but you had to manually change the code to do it. With the SmartDashboard, you will do it before a match starts without having to change any code. This is where you will add other functions to activate commands or display parameters in real time.

The implementation of smartDashboard we will be using is called Elastic. Elastic is automatically configured to look for a NetworkTables instance from the main robot. To connect to Elastic, go to the WPILib menu (Cntl+shift+P) and select 'Start Tool' and then 'Elastic'.

To start with a clean Elastic file, go to EDIT and Clear Layout and close any tabs if they exist.

Select Add Widget, Network Tables, SmartDashboard and then drag the SendableChooser to the grid. Close the Add Widget interface and save the layout. Now run autonomous and prove that you can use the Chooser to select the routine. You can add other autonomous commands and use the `m_chooser.addOption` to add them to the chooser widget.

To understand Elastic, we will do two exercises:

Exercise 1: In your new branch labeled with your name,

Task: Update your RomiBot2024 branch from remote Main

Task: Add a green LED to the LED subsystem

Task: Add Elastic buttons to turn green LED on and off. Test and debug your code. Save the Elastic layout to the RomiBot2024 project folder and commit.

The LED widgets should look something like this:

https://drive.google.com/file/d/10Bc3wSZY_XyLSvYRKKYvxxkxpk5Xp43OO/view?usp=drive_link

Exercise 2:

Task: Add a get method to the LEDSubsystem.

Task: Add a command to toggle the green LED.

Task: Use this get method to determine if the led needs to be turned on or off.

Task: Add a widget to Elastic to toggle the green LED.

Task: Test. Save. Commit

Exercise 3: Optional for extra credit

Task: Modify the ToggleLed command to pass the color parameter, so any of the three leds will toggle. Reference the LEDSet command.

At this point feel free to add other commands or displays on the dashboard. Save your layout.

Show off your good work to a mentor and let them know what you achieved and/or what you had trouble with.

Ask someone familiar with the competition robot to show you the driverstation and the dashboard with all the tabs that are used for the game.

When finished, commit and push to GitHub.